

Nama: Hans Malvin Djojo

<https://github.com/hansmalvin/LastMile>

Environment Setup

Create a project directory:

```
bash
mkdir express-crud-lab
cd express-crud-lab
```

Initialize a Node.js project:

```
bash
npm init -y
```

This command creates a package.json file with default settings.
Install required dependencies:

```
bash
npm install express mongoose body-parser
```

express: The Express.js framework for building web applications.
mongoose: An ODM for MongoDB, providing a higher-level abstraction for interacting with the database.
* body-parser: Middleware to parse incoming request bodies (JSON, URL-encoded).

Create the main application file:

Create a file named app.js in the project directory. This will be the entry point of your application.

Create a models directory:

```
bash
mkdir models
```

This directory will contain your Mongoose model definitions.
Create a route directory:

```
bash
mkdir routes
```

This directory will contain your Express routes.

Step-by-Step Instructions

Step 1: Connect to MongoDB

Add Mongoose connection code to app.js:

```
javascript
// app.js
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const app = express();
const port = 3000;

// Middleware to parse JSON request bodies
app.use(bodyParser.json());

// MongoDB connection string. Replace with your actual connection string.
const dbURI = 'mongodb://127.0.0.1:27017/express-crud-db'; // Replace with
your actual MongoDB connection string

mongoose.connect(dbURI, {
  useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => console.log('Connected to MongoDB'))
.catch(err => console.error('MongoDB connection error:', err));

app.listen(port, () => {
  console.log(Server is running on port ${port});
});
```

Explanation:

This code imports the necessary modules: express, mongoose, and body-parser.

It creates an Express application instance.

It sets up middleware to parse JSON request bodies.

It connects to the MongoDB database using Mongoose. Important: Replace 'mongodb://127.0.0.1:27017/express-crud-db' with the actual connection string for your MongoDB instance if it's different. Ensure MongoDB is running.

* It starts the Express server and listens on port 3000.

Run the application:

```
bash
node app.js
```

Expected Output: You should see "Connected to MongoDB" and "Server is running on port 3000" in the console if the connection is successful.

Step 2: Define a Mongoose Model

Create a model file `models/Book.js`:

```
javascript
// models/Book.js
const mongoose = require('mongoose');
const bookSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true
  },
  author: {
    type: String,
    required: true
  },
  genre: String,
  publishedYear: Number
});

module.exports = mongoose.model('Book', bookSchema);
```

Explanation:

This code defines a Mongoose schema called `bookSchema`.

The schema specifies the structure of a "Book" document in the MongoDB database.

It defines fields like `title`, `author`, `genre`, and `publishedYear`, along with their data types and validation rules (e.g., `title` and `author` are required).

It creates a Mongoose model named `'Book'` using the defined schema. This model will be used to interact with the "books" collection in the database.

Step 3: Create CRUD Routes

Create a routes file `routes/books.js`:

```
javascript
```

```
// routes/books.js
const express = require('express');
const router = express.Router();
const Book = require('../models/Book');
// GET all books
router.get('/', async (req, res) => {
  try {
    const books = await Book.find();
    res.json(books);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// GET a specific book by ID
router.get('/:id', async (req, res) => {
  try {
    const book = await Book.findById(req.params.id);
    if (!book) {
      return res.status(404).json({ message: 'Book not found' });
    }
    res.json(book);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// POST a new book
router.post('/', async (req, res) => {
  const book = new Book({
    title: req.body.title,
    author: req.body.author,
    genre: req.body.genre,
    publishedYear: req.body.publishedYear
  });

  try {
    const newBook = await book.save();
    res.status(201).json(newBook);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// PUT (update) an existing book
router.put('/:id', async (req, res) => {
  try {
```

```

const book = await Book.findByIdAndUpdate(req.params.id, req.body, { new: true
});
if (!book) {
return res.status(404).json({ message: 'Book not found' });
}
res.json(book);
} catch (err) {
res.status(400).json({ message: err.message });
}
});

// DELETE a book
router.delete('/:id', async (req, res) => {
try {
const book = await Book.findByIdAndDelete(req.params.id);
if (!book) {
return res.status(404).json({ message: 'Book not found' });
}
res.json({ message: 'Book deleted' });
} catch (err) {
res.status(500).json({ message: err.message });
}
});

module.exports = router;

```

Explanation:

This code defines Express routes for handling CRUD operations on the "books" resource.

It imports the Book model defined in models/Book.js.

GET /: Retrieves all books from the database using Book.find().

GET /:id: Retrieves a specific book by its ID using Book.findById().

POST /: Creates a new book using Book.save().

PUT /:id: Updates an existing book using Book.findByIdAndUpdate(). The { new: true } option ensures that the updated book is returned in the response.

DELETE /:id: Deletes a book using Book.findByIdAndDelete().

Error handling is included in each route to handle potential errors during database operations and return appropriate HTTP status codes and error messages.

Step 4: Mount the Routes in app.js

Update app.js to include the book routes:

```

javascript
// app.js (modified)

```

```

const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const bookRoutes = require('./routes/books'); // Import the book routes
const app = express();
const port = 3000;

app.use(bodyParser.json());

const dbURI = 'mongodb://127.0.0.1:27017/express-crud-db';

mongoose.connect(dbURI, {
  useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => console.log('Connected to MongoDB'))
.catch(err => console.error('MongoDB connection error:', err));

// Mount the book routes at the /books path
app.use('/books', bookRoutes);

app.listen(port, () => {
  console.log(Server is running on port ${port});
});

```

Explanation:

This code imports the bookRoutes from routes/books.js.

It mounts the bookRoutes at the /books path using app.use('/books', bookRoutes). This means that all routes defined in routes/books.js will be prefixed with /books. For example, the GET all books route will be accessible at /books/.

Step 5: Testing the API

Start the server:

```

bash
node app.js

```

Use curl or Postman to test the API endpoints:

- * Create a book (POST /books):

```

bash

```

```
curl -X POST -H "Content-Type: application/json" -d '{"title": "The Lord of the Rings", "author": "J.R.R. Tolkien", "genre": "Fantasy", "publishedYear": 1954}' http://localhost:3000/books
```

Expected Output: A JSON response containing the newly created book object, including its `_id` generated by MongoDB. Status code 201.

- * Get all books (GET /books):

```
bash
```

```
curl http://localhost:3000/books
```

Expected Output: A JSON array containing all book objects in the database.

- * Get a specific book (GET /books/:id): Replace `:id` with the actual ID of a book.

```
bash
```

```
curl http://localhost:3000/books/654321abcdef0123456789
```

Expected Output: A JSON object representing the book with the specified ID. If the ID is invalid or the book doesn't exist, you'll get a 404 error.

- * Update a book (PUT /books/:id): Replace `:id` with the actual ID of a book.

```
bash
```

```
curl -X PUT -H "Content-Type: application/json" -d '{"title": "The Hobbit", "author": "J.R.R. Tolkien", "publishedYear": 1937}' http://localhost:3000/books/654321abcdef0123456789
```

Expected Output: A JSON response containing the updated book object.

- * Delete a book (DELETE /books/:id): Replace `:id` with the actual ID of a book.

```
bash
```

```
curl -X DELETE http://localhost:3000/books/654321abcdef0123456789
```

Expected Output: A JSON response with the message "Book deleted".

Answer:

① README.md M JS app.js U X

express-crud-lab > JS app.js > ...
1 const express = require('express');
2 const mongoose = require('mongoose');
3 const bodyParser = require('body-parser');
4
5 const app = express();
6 const port = 3000;
7
8 app.use(bodyParser.json());
9
10 mongoose.connect('mongodb://127.0.0.1:27017/express-crud-db')
11 .then(() => console.log('Connected to MongoDB'))
12 .catch(err => console.error(err));
13
14
15 app.listen(port, () => {
16 console.log(`Server running on port \${port}`);
17 });
18

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - express-crud-lab ⚠ + ▢ 🗑
PS C:\Users\ASUS\LastMile\express-crud-lab> node app.js
at Object.<anonymous> (C:\Users\ASUS\LastMile\express-crud-lab\app.js:4:20)
at Module._compile (node:internal/modules/cjs/loader:1812:14)
at Object..js (node:internal/modules/cjs/loader:1943:10) {
code: 'MODULE_NOT_FOUND',
requireStack: ['C:\Users\ASUS\LastMile\express-crud-lab\app.js']
}

Node.js v24.14.0
PS C:\Users\ASUS\LastMile\express-crud-lab> node app.js
Server running on port 3000
Connected to MongoDB

① README.md M X JS app.js U JS Book.js U JS books.js U localhost:3000 X

POST http://localhost:3000/books Send
Query Headers 2 Auth Body 1 Tests Pre Run
JSON XML Text Form Form-encode GraphQL Binary
JSON Content Format
1 [{"title": "wow", "author": "J.R.R. Tolkien", "genre": "Fantasy", "publishedYear": 1954}]

Status: 201 Created Size: 121 Bytes Time: 5 ms
Response Headers 6 Cookies Results Docs </> ≡
1 {
2 "title": "wow",
3 "author": "J.R.R. Tolkien",
4 "genre": "Fantasy",
5 "publishedYear": 1954,
6 "_id": "69fde4bfa85809c4dc7e231",
7 "_v": 0
8 }

① README.md M JS app.js U JS Book.js U JS books.js U localhost:3000 X

GET http://localhost:3000/books Send
Query Headers 2 Auth Body 1 Tests Pre Run
JSON XML Text Form Form-encode GraphQL Binary
JSON Content Format
1 [{"title": "wow", "author": "J.R.R. Tolkien", "genre": "Fantasy", "publishedYear": 1954}]

Status: 200 OK Size: 263 Bytes Time: 10 ms
Response Headers 6 Cookies Results Docs </> ≡
1 [
2 {
3 "_id": "69fde4b2fa85809c4dc7e230",
4 "title": "The Lord of the Rings",
5 "author": "J.R.R. Tolkien",
6 "genre": "Fantasy",
7 "publishedYear": 1954,
8 "_v": 0
9 },
10 {
11 "_id": "69fde4bfa85809c4dc7e231",
12 "title": "wow",
13 "author": "J.R.R. Tolkien",
14 "genre": "Fantasy",
15 "publishedYear": 1954,
16 "_v": 0
17 }
18]

GET

http://localhost:3000/books/69fde4b2fa85809c4dc7e230

Send

Query

Headers 2

Auth

Body 1

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1 { "title": "wow", "author": "J.R.R. Tolkien", "genre": "Fantasy", "publishedYear": 1954 }

PUT

http://localhost:3000/books/69fde4b2fa85809c4dc7e230

Send

Query

Headers 2

Auth

Body 1

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1 { "title": "The Hobbit", "author": "J.R.R. Tolkien", "publishedYear": 1937 }

DELETE

http://localhost:3000/books/69fde4b2fa85809c4dc7e230

Send

Query

Headers 2

Auth

Body 1

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1 { "title": "The Hobbit", "author": "J.R.R. Tolkien", "publishedYear": 1937 }

Status: 200 OK

Size: 139 Bytes

Time: 7 ms

Response

Headers 6

Cookies

Results

Docs

</>

≡

1 {

2 " id": "69fde4b2fa85809c4dc7e230",

3 "title": "The Lord of the Rings",

4 "author": "J.R.R. Tolkien",

5 "genre": "Fantasy",

6 "publishedYear": 1954,

7 "__v": 0

8 }

Status: 200 OK

Size: 128 Bytes

Time: 14 ms

Response

Headers 6

Cookies

Results

Docs

</>

≡

1 {

2 " id": "69fde4b2fa85809c4dc7e230",

3 "title": "The Hobbit",

4 "author": "J.R.R. Tolkien",

5 "genre": "Fantasy",

6 "publishedYear": 1937,

7 "__v": 0

8 }

Status: 200 OK

Size: 26 Bytes

Time: 7 ms

Response

Headers 6

Cookies

Results

Docs

</>

≡

1 {

2 "message": "Book deleted"

3 }